EVNet Sentinel: A Leakage-Corrected Reproduction and Comparative Study of Intrusion Detection on Electric Vehicle Charging Networks

Mukta Varak
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0013

Shruti Chaurasiya
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0018

Shardul Chogale*
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0021

Neha Chavhan
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0022

Mahesh Khandke
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
makhandke@gmail.com

***Abstract*—Public electric vehicle chargers are networked computers, and an attack on their link to the central management system can stop charging and billing. Machine-learning intrusion detectors evaluated on the CICEVSE2024 dataset report accuracies near 99 percent. This study reproduces the most recent of them, an online Adaptive Random Forest with ADWIN drift detection, and audits whether that accuracy reflects attack detection. We follow a quantitative, computational design: the published preprocessing is replayed, every feature is probed for label leakage, and six static classifiers, including the gradient-boosted XGBoost and LightGBM, and the online model are retrained on corrected data and evaluated over three seeds with macro-F1. A single capture timestamp recovers the attack label with 99.76 percent accuracy, above the published 98.40 percent, and the drift detector fires on the joins between capture files rather than on changes in traffic. After correction, flooding attacks remain near-perfectly separable while six network-scan types are not, and a mutual-information analysis shows that this limit lies in the flow features rather than in the models. A per-attack test finds that no attack passes as normal traffic; the tree ensembles are statistically tied, and XGBoost is selected as the primary detector by its marginally lower false-alarm rate. The work is delivered as EVNet Sentinel, with a prediction service, role-based response guidance and an interactive web front end.**

***Index Terms*—Intrusion detection, electric vehicle charging, CICEVSE2024, data leakage, adaptive random forest, concept drift, OCPP, machine learning evaluation**

I. Introduction

Electric mobility depends on a growing network of public charging stations (Electric Vehicle Supply Equipment, EVSE). Each station talks to the vehicle over ISO 15118 and to a Charging Station Management System (CSMS) over the Open Charge Point Protocol (OCPP) [14]. The OCPP link carries authorisation, metering and remote commands, so an attacker who floods or probes a station can interrupt charging and the revenue that depends on it [1], [3]. Detecting such attacks on the station network is therefore a practical security need.

Most detectors for this setting are machine-learning classifiers trained on the CICEVSE2024 dataset [8], and reported accuracies are close to perfect [1], [2]. Makhmudov et al. [4] proposed the first online detector for charging networks, combining an Adaptive Random Forest (ARF) [6] with ADaptive WINdowing (ADWIN) [7] so the model can adapt as traffic changes, a problem known as concept drift [15]. They report 98.40% multiclass accuracy.

Near-perfect scores on a public benchmark deserve scrutiny. Security datasets are often recorded one attack per capture, and features that identify the recording rather than the traffic can leak the label [10], [11], [13]. **The research problem** addressed here is whether the published accuracy measures attack detection or a property of how CICEVSE2024 was assembled. The problem is significant because an operator who trusts an inflated score will deploy a detector that fails on the attacks that matter.

Objectives. The study aims to (i) reproduce the published pipeline and test it for leakage; (ii) report corrected, multi-seed benchmarks of static and online models; (iii) explain which attacks remain hard and why; (iv) choose a primary detector per attack from detection, naming and false-alarm rates; and (v) deliver a usable system around the corrected models.

Research questions. The study is organised around four questions:

- **RQ1:** Does the published preprocessing leak the label?
- **RQ2:** Is the reported drift real?
- **RQ3:** What accuracy remains once the leaks are removed, and which attacks are hard?

*Corresponding author: Shardul Chogale (shardulchogale1983@gmail.com).

- **RQ4:** Which model should an operator trust, and for which attack?

Contributions. We present EVNet Sentinel, a passive network intrusion detection system for the station network, and use it to deliver five verifiable leakage findings, corrected benchmarks, a model-independent explanation of the reconnaissance ceiling, a per-attack evaluation with a confidence-gated naming rule, and a working prediction service and web front end.

II. Literature Review

Surveys and threat analyses. Bean et al. [1] survey machine-learning intrusion detection for charging equipment across network, host and power data and review 24 studies on CICEVSE2024; they identify concept drift and OCPP-level monitoring as open problems but take reported accuracies as given. Malimage et al. [3] analyse real incidents, protocol weaknesses (OCPP, OCPI) and standards such as ISO/SAE 21434, motivating detection without proposing a detector.

Detectors on CICEVSE2024. Joglekar et al. [2] combine a network detector (XGBoost, 99.99% accuracy) with a host detector. They remove IP and MAC identifiers but keep timestamps, and train offline. Makhmudov et al. [4] stream the data through ARF+ADWIN with 20 trees and $\delta = 0.002$, reporting 99.13% binary and 98.40% multiclass accuracy, 11 drift events and 3.7 ms per flow; drift handling is offered as the main advantage. Tanyildiz et al. [5] instead predict the time to the next attack with a GAN-augmented regressor on a different dataset.

Online learning and drift. Gama et al. [15] distinguish sudden, gradual and recurring drift and stress that a detector should be judged against the change it is meant to find. ARF [6] trains each tree with its own drift detector and replaces it with a background tree when drift is signalled. ADWIN [7] keeps a variable-length window of recent errors and cuts it when two sub-windows differ significantly. River [9] provides both. Random Forest [16], the strongest static model in most flow-based studies, serves as the offline reference.

Evaluation pitfalls. Kaufman et al. [10] formalise leakage as information in the training data that would not be available at prediction time. Arp et al. [11] list sampling bias, spurious correlations and inappropriate baselines among the most common errors in security machine learning, and Sommer and Paxson [13] warn that intrusion detectors often learn artefacts of the evaluation environment.

Research gap. Table I shows that no prior CICEVSE2024 study tests its preprocessing for capture-level leakage, reports results over several seeds, uses a class-balanced metric as the headline, or checks whether detected drift is real. This study addresses all four gaps and adds a per-attack, operator-oriented evaluation.

III. Proposed Methodology and Architecture

Fig. 1 summarises the method and the delivered system, from capture to response.

TABLE I
Comparison of related work with this study

Work	Approach	Reported	Audit
Bean [1]	Survey (24 studies)	—	×
Joglekar [2]	XGBoost + host IDS, offline	99.99% acc.	IP/MAC
Malimage [3]	Threat analysis	—	—
Makhmudov [4]	ARF+ADWIN, online	98.40% acc.	×
Tanyildiz [5]	GAN time-to-attack	$R^2 = 0.74$	—
This work	**Audit + 4 models + ARF, 3 seeds**	**0.558 macro-F1**	✓

A. Research Design

The study is *quantitative and computational*, in two parts. The first is a *reproduction audit*: the published pipeline is replayed exactly and each suspected artefact is isolated with a controlled experiment that changes one factor at a time (a feature, the stream order, or the split). The second is a *comparative experiment*: six static classifiers and the online model are trained on corrected data under one protocol. This design suits the objectives because each finding becomes a measurable, repeatable test rather than an opinion, and every number can be regenerated from the public code.

The detector is *passive*. It reads a mirrored copy of the switch traffic, classifies each completed network flow and raises an alert, but never blocks traffic. This matches how CICEVSE2024 itself was recorded.

B. Data

CICEVSE2024 [8] was recorded by the Canadian Institute for Cybersecurity on a testbed with two charging stations (EVSE-A and EVSE-B), a local and a remote CSMS, and a vehicle controller, with stations either idle or charging. Network flows were extracted with NFStream into 86 features and labelled from the capture file name. The 59 capture files hold 2,744,700 flows in 15 classes:

- five *volumetric floods*: SYN, TCP, UDP, PSH-ACK and SynonymousIP;
- six *nmap reconnaissance modes*: TCP port scan, SYN stealth scan, OS fingerprinting, service-version detection, vulnerability scan and aggressive scan;
- three *low-rate attacks*: ICMP flood, ICMP fragmentation and Slowloris;
- benign traffic.

The dataset lists its devices but not which device attacked which. Because every flow carries source and destination MAC addresses, we derived the route of each attack capture (Table II). Every attack targets a charging station, and in six captures the compromised vehicle scans EVSE-B over the charging cable.

The classes are highly imbalanced: after deduplication the four large floods hold about 196,000–260,000 flows each, the scans 28,000–39,000, Slowloris 2,721, and benign traffic only 81.

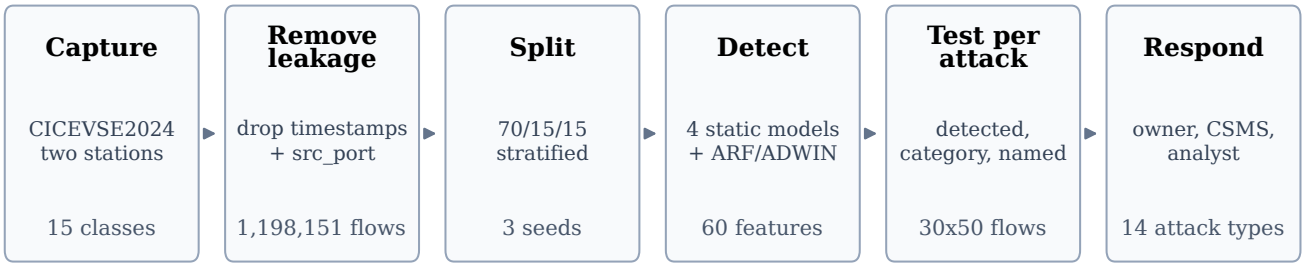


Fig. 1. Method and system at a glance, from capture to response.

TABLE II
ATTACK ROUTES DERIVED FROM MAC ADDRESSES (57 ATTACK CAPTURES)

Attacker	Target	Caps.	Attacks
Kali PC (Wi-Fi)	EVSE-A	24	all except UDP flood
Kali PC (Wi-Fi)	EVSE-B	23	all except UDP flood, Slowloris
Raspberry Pi (Wi-Fi)	both	4	UDP flood
Compromised vehicle	EVSE-B	6	the six scans

C. Materials and Tools

All experiments ran on one Apple M5 laptop (10 cores, 16 GB RAM) with pinned software versions:

- **Analysis:** Python 3.13, NumPy 2.4, pandas and scikit-learn 1.8 [12] for the static models, and River 0.26 [9] for ARF+ADWIN.
- **Service:** a FastAPI prediction service that loads every model and its scaler at start-up.
- **Front end:** a Next.js 16 web application.
- **Reporting:** matplotlib for every figure, drawn from the same exported data the front end displays.

The dataset, code and exported results are public (Section VI).

D. Procedure

Step 1, replay. The reference preprocessing [4] was rerun verbatim to reproduce its feature set.

Step 2, leakage probes. A depth-25 decision tree was trained on each feature alone and scored by 3-fold cross-validated accuracy. Keeping or dropping each suspicious column was crossed before deduplication.

Step 3, corrected preprocessing. All captures are concatenated and labelled. Identifiers (flow ID, IP, MAC, vendor) and constant columns are removed. The six absolute timestamps (`*_first_seen_ms`, `*_last_seen_ms`) and `src_port` are dropped, while the destination port is kept. Duplicate rows are removed, the data are split 70/15/15 with class stratification, and each feature is standardised with training statistics only. This leaves 1,198,151 flows and 60 features (838,705 / 179,723 / 179,723 rows).

Step 4, training. The static models are:

- Random Forest [16]: 100 trees, depth 15;
- Decision Tree: depth 20;
- XGBoost: up to 400 rounds, depth 8, learning rate 0.1;
- LightGBM: up to 600 rounds, 63 leaves, learning rate 0.05;
- Logistic Regression and a linear SVM, both trained by stochastic gradient descent.

All six use balanced class weights $w_k = N/(Kn_k)$ (as sample weights for the boosted models), so that rare classes count as much as common ones. The boosted models stop early on the validation split, after 110–125 rounds; their settings were fixed in advance, not tuned on the test set. ARF uses the published configuration: 20 trees, half the features per split, a grace period of 30, and ADWIN with $\delta = 0.002$. It is run prequentially, predicting each flow before learning from it, in both capture order and shuffled order.

Step 5, per-attack testing. For each attack, model and seed, 30 batches of 50 unseen flows are scored. Mixed campaigns of four simultaneous attacks are also run. Regression gates fail the run if detection of any attack falls below 0.95.

Step 6, delivery. The models are served by the API, and the exported results drive the web dashboard and a simulation that replays held-out flows along their recorded routes.

E. Data Analysis

Metrics. For class k with precision P_k and recall R_k,

$$F_{1,k} = \frac{2P_kR_k}{P_k + R_k}, \qquad \text{macro-}F_1 = \frac{1}{K}\sum_{k=1}^{K} F_{1,k}. \tag{1}$$

Macro-F_1 weights the 15 classes equally. Accuracy, by contrast, is dominated by the floods. Every figure is the mean and standard deviation over seeds 42, 1337 and 2024, where each seed changes both the split and the model initialisation.

Stream order. Prequential accuracy is compared with the trivial rule "repeat the previous label". That rule's accuracy equals the label autocorrelation

$$\rho = \Pr\left[y_t = y_{t-1}\right], \tag{2}$$

which any model on an ordered stream must beat before it can be said to learn. For a shuffled stream $\rho = \sum_k p_k^2$, where p_k is the share of class k.

ADWIN. ADWIN signals drift when, for some split of its window W into W_0W_1, the mean error of the two parts differs by more than

$$\epsilon_{\text{cut}} = \sqrt{\frac{1}{2m}\ln\frac{4n}{\delta}}, \qquad m = \frac{1}{1/n_0 + 1/n_1}, \tag{3}$$

where n_0 and n_1 are the sizes of the two parts and $n = |W|$. In capture order every file boundary is an abrupt change by construction, so we test whether the alarms fall on them.

Drift alignment. For each ADWIN alarm e_i, we measure the distance to the nearest capture-file boundary, $d_i = \min_j |e_i - b_j|$. We count the alarms within $w = 100$ flows of a boundary and compare that count H with the counts $H^{(r)}$ from $R = 2000$ random placements of the same number of alarms:

$$p = \frac{1 + \sum_{r=1}^{R} \mathbb{I}\left[H^{(r)} \geq H\right]}{1 + R}. \tag{4}$$

Per-feature separability. Within one category g (DoS or reconnaissance), we measure how much of the class uncertainty a single feature X_j removes:

$$S_j^{(g)} = \frac{I\left(X_j; Y \mid g\right)}{H\left(Y \mid g\right)}, \tag{5}$$

estimated with a k-nearest-neighbour mutual-information estimator [17] on 6,000 held-out flows per category. S_j does not depend on any model, so it describes the data itself.

Per-attack rates. For each batch we measure three rates: the share detected (called any attack), the share placed in the right category, and the share named exactly. On benign flows we measure the false-alarm rate. The primary detector is the model with the fewest false alarms among those whose mean exact-naming rate is within 0.02 of the best.

Confidence gating. A model names an attack only if its top class probability is at least t:

$$\text{output}(\mathbf{x}) = \begin{cases} \arg\max_k \hat{p}(k \mid \mathbf{x}) & \text{if } \max_k \hat{p}(k \mid \mathbf{x}) \geq t, \\ \text{category of that class} & \text{otherwise.} \end{cases} \tag{6}$$

IV. Results

A. Leakage in the published pipeline

Table IV lists the leakage probes. A decision tree on `bidirectional_first_seen_ms` alone recovers the 15-class label with 0.9976 accuracy, above the 0.9840 reported for the full published model. The reference preprocessing keeps all six timestamp columns. It also drops `dst_port`, the feature that defines a port scan. The source port, which the operating system allocates almost sequentially, alone recovers 0.4028.

Table III shows why the published configuration is fragile. Dropping the destination port while keeping the timestamps leaves 273,932 reconnaissance flows after deduplication. These flows can then be separated by *when* they were captured rather than by what they did. Removing both the timestamps and the destination port collapses the scans to 6,821 flows: almost every remaining scan flow becomes a duplicate.

TABLE III
FLOWS LEFT AFTER DEDUPLICATION, BY KEPT OR DROPPED COLUMN

Timestamps	`dst_port`	Total	Recon.
kept	kept	2,728,799	1,724,504
kept	**dropped (published)**	**1,277,520**	**273,932**
dropped	kept (ours)	1,198,143	194,737
dropped	dropped	977,862	6,821

TABLE IV
LABEL RECOVERED FROM A SINGLE FEATURE (15 CLASSES, 3-FOLD CV)

Feature	Nature	Accuracy
`bidirectional_first_seen_ms`	fingerprint	**0.9976**
`src_port`	partial	0.4028
`dst_port`	legitimate	0.2917
Published multiclass accuracy [4]		*0.9840*

TABLE V
ARF+ADWIN PREQUENTIAL ACCURACY BY STREAM ORDER (LEAK-FREE)

Order	ARF acc.	Repeat-last ρ	Drift alarms
Capture order	0.9997	0.99997	43
Shuffled	0.5585	0.117	14

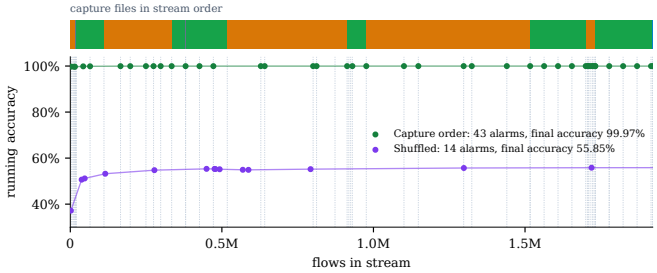


Fig. 2. ADWIN alarms over the training stream. Top band: the 58 capture files in stream order. Dashed lines mark file boundaries.

B. Stream order and drift

In capture order each file holds a single class, so over 1,921,182 training flows the label changes only 54 times and $\rho = 0.99997$ (Table V). ARF+ADWIN reaches 0.9997, which does not beat the repeat-last rule. On a shuffled stream it reaches 0.5585.

Of the 43 drift alarms in capture order, 42 lie within 100 flows of a file boundary, with a median distance of 22 flows. Random placement would put only 2.2 ± 1.5 alarms that close ($p < 0.0005$), as Fig. 2 shows.

C. Corrected benchmarks

Table VII gives the corrected results. The four tree-based models, Random Forest, Decision Tree, XGBoost and LightGBM, are within one standard deviation of each other: gradient boosting does not break the ceiling. The linear models trail by more than 0.1 macro-F_1, and all six reach about 0.86 accuracy. On a like-for-like split, removing the timestamps lowers Random Forest's macro-F_1 from 0.9547 to 0.5894,

TABLE VI
EFFECT OF REMOVING SRC_PORT (MACRO-F_1, SINGLE SEED)

Model	With	Without	Δ
Random Forest	0.5894	0.5138	−0.076
Decision Tree	0.6834	0.5135	**−0.170**

TABLE VII
MULTICLASS RESULTS, MEAN ± STD OVER 3 SEEDS (CORRECTED FEATURES)

Model	Macro-F_1	Acc.	Fit (s)
Random Forest	**0.558±0.008**	0.868	8.5
Decision Tree	0.557±0.010	0.868	3.4
XGBoost	0.551±0.011	0.869	33.0
LightGBM	0.550±0.004	0.868	42.4
Logistic Regression	0.436±0.017	0.863	78.9
Linear SVM	0.411±0.017	0.862	34.9

TABLE VIII
PER-CLASS F_1, RANDOM FOREST, MEAN ± STD OVER 3 SEEDS

Class	Support	F_1
SYN, TCP, PSH-ACK, SynIP	196k–259k	0.9999±0.0000
UDP flood	32,072	0.9994±0.0001
TCP port scan	35,123	0.265
OS fingerprinting	27,702	0.223
Vulnerability scan	39,002	0.204
Service-version detection	30,338	0.156
Aggressive scan	27,815	0.145
SYN stealth scan	34,765	0.105
Benign	81	0.986
ICMP flood / ICMP fragmentation	32 / 28	0.660 / 0.439
Slowloris	2,721	0.191

while the SVM changes by only +0.03. Removing `src_port`, with the rows and the split held fixed, costs the deeper Decision Tree twice as much as Random Forest (Table VI). Extra depth was buying finer bands of port numbers, so the tree's earlier advantage was memorisation.

D. Per-class results and separability

Table VIII shows a sharp split. The five floods reach $F_1 \geq 0.999$ in every seed. The six scans average 0.183, and sample size does not explain the gap: UDP flood (32,072 flows) scores 0.9994, while TCP port scan (35,123 flows) scores 0.265. The separability measure (Fig. 4) shows the same split. Within DoS, one inter-arrival-time feature resolves 98% of the class uncertainty. Within reconnaissance, the best feature resolves 5%.

The errors among the scans follow a pattern (Fig. 3). Random Forest labels 32–48% of every other scan as OS fingerprinting, so the scans collapse into one "sink" class. A further 8–11% of each scan is labelled Slowloris, because both hold long-lived, nearly idle connections. This confusion accounts for almost all of the error between the reconnaissance and DoS categories.

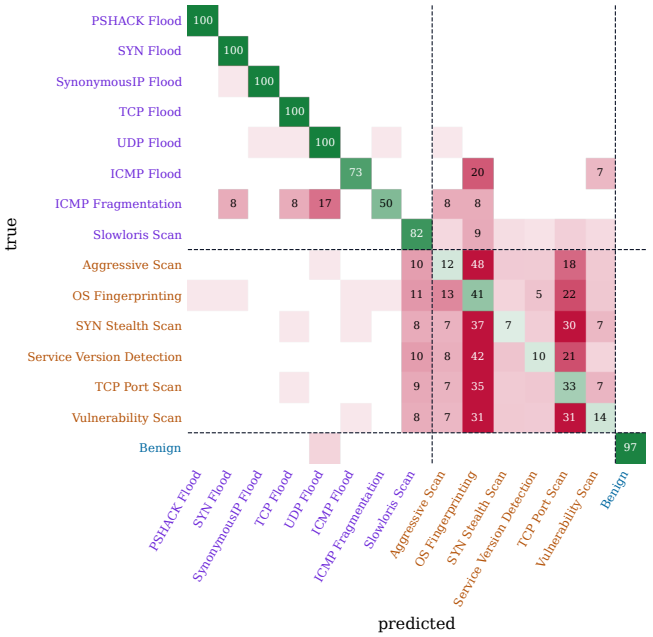


Fig. 3. Random Forest confusion matrix, row-normalised and averaged over three seeds (%; cells under 5% unlabelled). Dashed lines separate DoS, reconnaissance and benign.

TABLE IX
EXACT-NAMING RATE E PER ATTACK FOR RANDOM FOREST, THE BEST MODEL AT NAMING, AND THE BEST CATEGORY RATE C

Attack	RF E	Best E	Best C
Five volumetric floods	1.00	all ($\geq$ 0.98)	1.00
Slowloris	0.82	XGB 0.83	0.83
TCP port scan	0.33	LR 0.66	0.97
OS fingerprinting	0.42	XGB 0.43	0.95
SYN stealth scan	0.08	SVM 0.43	0.97
Vulnerability scan	0.14	SVM 0.26	0.97
Aggressive scan	0.12	LGBM 0.17	0.94
Service-version det.	0.10	DT 0.13	0.97

E. Per-attack detection and confidence

Table IX lists the exact-naming rate per attack. Across 90 mixed campaigns, every model flags at least 99.9% of attack flows as malicious. Every model names the floods exactly ($\geq$ 0.98). The scans are placed in the right category 96% of the time by the best model for each, but named exactly only 35% of the time. Table XI gives the false-alarm rates on benign traffic. By the selection rule of Section III, XGBoost is the primary detector; its lead over Random Forest (0.02 against 0.03) is within the 95% intervals of the 12 benign test flows, so the two are interchangeable in practice. Applying the confidence gate (6) at $t = 0.5$ cuts Random Forest's wrong scan names from 81% to 3% and reports 94% of scans by category, while 93% of DoS flows are still named exactly (Table X).

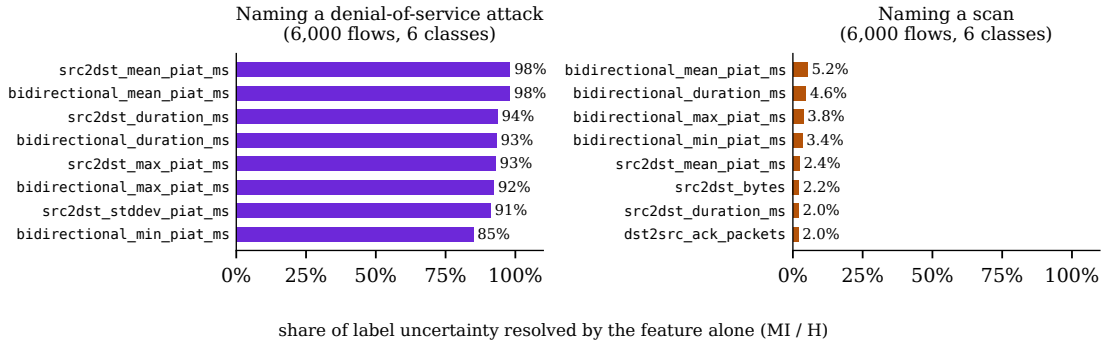


Fig. 4. Per-feature separability S_j of (5) within DoS and within reconnaissance flows (6,000 held-out flows each).

TABLE X
CONFIDENCE-GATED NAMING FOR RANDOM FOREST: SHARE NAMED RIGHT (R), NAMED WRONG (W), REPORTED BY CATEGORY ONLY (H)

	DoS			**Reconnaissance**		
t	R	W	H	R	W	H
0	0.97	0.03	0.00	0.19	0.81	0.00
0.5	0.93	0.00	0.07	0.03	0.03	0.94
0.9	0.87	0.00	0.13	0.01	0.00	0.99

TABLE XI
FALSE-ALARM RATE ON BENIGN TRAFFIC BY MODEL

	RF	**DT**	**XGB**	**LGBM**	**LR**	**SVM**
False alarms	0.03	0.11	**0.02**	0.03	0.44	0.58

V. DISCUSSION

What the published accuracy measures. Each attack was recorded in its own file at its own time, so an absolute timestamp identifies the recording, and through it the label. This is leakage in the sense of Kaufman et al. [10]: the feature would not carry this information on live traffic. Trees exploit it and linear models cannot, which is why removing the timestamps costs Random Forest 0.365 macro-F_1 but leaves the SVM almost unchanged. Much of the reported gap between tree and linear models in earlier work therefore reflects how well each model could use the leak.

Drift. The online model's capture-order accuracy equals the repeat-last baseline, and its drift alarms coincide with file boundaries. Both observations point the same way: the stream changes where files were joined, not because charging traffic evolved. Drift handling is presented in [4] as the method's advantage, but CICEVSE2024 contains no real drift with which to test that claim. This matches the warnings of Arp et al. [11] about inappropriate baselines and of Sommer and Paxson [13] about learning artefacts of the evaluation environment.

Floods and scans. The near-perfect flood scores are plausible rather than leaked. The fingerprint columns are removed; a flood has a strong physical signature that a single timing feature captures; and the linear models also name the floods. We nevertheless claim this only for this testbed: the random split can place near-identical flows from one capture on both sides of the split, and floods sent at a lower rate would look different. The scans are a different matter. All six are modes of one tool, the aggressive mode includes two of the others, and Fig. 4 shows that no single flow statistic separates them. More model capacity cannot recover information the features do not contain.

Operational meaning. No attack passes as normal traffic. The weakness is *naming*, not *noticing*. Random Forest, XGBoost and LightGBM name attacks equally well and raise few false alarms; XGBoost is selected by the pre-stated rule, but Random Forest trains four times faster and is an equally sound choice. For scans, a confidence gate turns confidently wrong names into correct category alerts, which is the information an operator can act on.

Limitations. Labels come from file names, so background traffic recorded during an attack carries the attack's label. Only 81 benign flows exist, so false-alarm rates are indicative. The headline results use a random rather than a grouped split. The study covers a single testbed with tools at default settings, and the host and power data in CICEVSE2024 are not used.

VI. CONCLUSION

This study set out to test whether the high accuracies reported on CICEVSE2024 measure attack detection, and found that they largely do not:

- one capture timestamp beats the published model;
- streaming accuracy in capture order is matched by repeating the previous label;
- the drift detector responds to file boundaries.

With these effects removed, the corrected benchmark is 0.558 macro-F_1 for Random Forest over three seeds. Volumetric floods remain near-perfectly separable on this testbed, while the six nmap scan modes are not, and a model-independent analysis places that limit in the flow features. Gradient-boosted trees (XGBoost, LightGBM) tie with Random Forest rather than surpass it. Operationally, every attack is detected, XGBoost is the recommended detector (statistically tied with Random Forest), and requiring confidence before naming an attack replaces most wrong scan names with correct category alerts.

The system, code and results are public at https://github.com/Mukta01/EVNet_Sentinel.

VII. Future Work and Recommendations

1) **Grouped evaluation.** Report results with whole capture files held out, which removes near-duplicate flows from both sides of the split.
2) **Sequence features.** Model the packet sequence within each flow, which flow statistics discard and which may separate the scan modes.
3) **More modalities.** Fuse the host-event and power traces in CICEVSE2024 with the network flows.
4) **Stronger baselines for online learning.** Evaluate every streaming model against the repeat-last rule (2) on grouped streams.
5) **Response.** Measure the response guidance itself, for example a quarantine of the attacking device approved by an operator, on a live testbed.
6) **Recommendation for dataset builders.** Release per-capture identifiers and benign traffic in volume, so that leakage can be checked and false alarms measured.

Acknowledgment

We build on the open-source implementation of [4] and thank the Canadian Institute for Cybersecurity for the CICEVSE2024 dataset.

References

[1] J. Bean, V. Manias et al., "Cybersecurity of electric vehicle charging infrastructure: Recent advances, open challenges, and future directions," arXiv:2605.24190v1, May 2026.
[2] C. Joglekar et al., "A hybrid intrusion detection system for electric vehicle charging infrastructure," arXiv:2606.23236v1, 2026.
[3] K. Malimage et al., "Cybersecurity challenges in the electric vehicle market," in *Proc. CIGRE Canada Conf.*, 2023, pp. 1–6.
[4] F. Makhmudov, D. Kilichev, U. Giyosov, and F. Akhmedov, "Online machine learning for intrusion detection in electric vehicle charging systems," *Mathematics*, vol. 13, no. 5, p. 712, 2025, doi: 10.3390/math13050712.
[5] E. Tanyildiz et al., "Detection of cyber attacks in electric vehicle charging systems using a remaining useful life generative adversarial network," *Scientific Reports*, vol. 15, 2025.
[6] H. M. Gomes et al., "Adaptive random forests for evolving data stream classification," *Machine Learning*, vol. 106, no. 9–10, pp. 1469–1495, 2017.
[7] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining*, 2007, pp. 443–448.
[8] Canadian Institute for Cybersecurity, "CICEVSE2024 dataset," University of New Brunswick, 2024. [Online]. Available: https://www.unb.ca/cic/datasets/
[9] J. Montiel et al., "River: Machine learning for streaming data in Python," *J. Mach. Learn. Res.*, vol. 22, no. 110, pp. 1–8, 2021.
[10] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012.
[11] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971–3988.
[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[13] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305–316.
[14] Open Charge Alliance, "Open Charge Point Protocol 2.0.1," 2020. [Online]. Available: https://openchargealliance.org/protocols/open-charge-point-protocol/
[15] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.
[16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[17] A. Kraskov, H. Stögbauer, and P. Grassberger, "Estimating mutual information," *Physical Review E*, vol. 69, no. 6, p. 066138, 2004.